

Andrej Karpathy Killed RAG. Or Did He? The LLM Wiki Pattern

5,000 stars in 48 hours. A GitHub Gist. No code. Just an idea file. And the entire AI community lost its collective mind.



Mandar Karhade, MD. PhD.

Following ▾

14 min read · 5 days ago

TLDR

- Andrej Karpathy published a GitHub Gist describing “LLM Wiki,” a pattern where the LLM builds and maintains a persistent, compounding markdown knowledge base instead of re-retrieving documents on every query like traditional RAG.
- The architecture has three layers: raw sources (immutable), a wiki (LLM-maintained markdown with cross-references), and a schema (configuration directing agent behavior). No vector database required at personal scale.
- The community is split: half thinks RAG is officially dead, the other half thinks Karpathy just gave a fancy name to a cache layer. Both sides have a point.
- The real insight isn’t “RAG bad, wiki good.” It’s that knowledge should compound, not evaporate. And LLMs are finally good enough at bookkeeping to make that practical.

- Enterprise scalability is the elephant in the room: no RBAC, no ACID transactions, no concurrency controls. This is a personal knowledge weapon, not an enterprise platform. Yet.

Free Link for everyone: Clap 50, Follow and Subscribe to me. Follow the **publication**. Join Medium to support other writers too! Cheers

Please subscribe to my new profile <https://medium.com/@ThisWorld> where I am covering Health tech, Global tech, and AI Governance through multi-part deep investigative articles.

Link to the Gist :

LLM Wiki

A pattern for building personal knowledge bases using LLMs.

This is an idea file, it is designed to be copy pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / Pi, or etc.). Its goal is to communicate the high level idea, but your agent will build out the specifics in collaboration with you.

The core idea

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM **incrementally builds and maintains a persistent wiki** — a structured, interlinked collection of markdown files that sits between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then *kept current*, not re-derived on every query. This is the key difference: **the wiki is a persistent, compounding artifact**. The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

You never (or rarely) write the wiki yourself — the LLM writes and maintains all of it. You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time. In practice, I have the LLM agent open on one side and Obsidian open on the other. The LLM makes edits based on our conversation, and I browse the results in real time — following links, checking the graph view, reading the updated pages. Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase.

This can apply to a lot of different contexts. A few examples:

- **Personal:** tracking your own goals, health, psychology, self-improvement — filing journal entries, articles, podcast notes, and building up a structured picture of yourself over time.
- **Research:** going deep on a topic over weeks or months — reading papers, articles, reports, and incrementally building a comprehensive wiki with an evolving thesis.
- **Reading a book:** filing each chapter as you go, building out pages for characters, themes, plot threads, and how they connect. By the end you have a rich companion wiki. Think of fan wikis like [Tolkien Gateway](#) — thousands of interlinked pages covering characters, places, events, languages, built by a community of volunteers over years. You could build

places, events, languages, built by a community of volunteers over years. You could build something like that personally as you read, with the LLM doing all the cross-referencing and maintenance.

- **Business/team:** an internal wiki maintained by LLMs, fed by Slack threads, meeting transcripts, project documents, customer calls. Possibly with humans in the loop reviewing updates. The wiki stays current because the LLM does the maintenance that no one on the team wants to do.
- **Competitive analysis, due diligence, trip planning, course notes, hobby deep-dives** — anything where you're accumulating knowledge over time and want it organized rather than scattered.

Architecture

There are three layers:

Raw sources — your curated collection of source documents. Articles, papers, images, data files. These are immutable — the LLM reads from them but never modifies them. This is your source of truth.

The wiki — a directory of LLM-generated markdown files. Summaries, entity pages, concept pages, comparisons, an overview, a synthesis. The LLM owns this layer entirely. It creates pages, updates them when new sources arrive, maintains cross-references, and keeps everything consistent. You read it; the LLM writes it.

The schema — a document (e.g. CLAUDE.md for Claude Code or AGENTS.md for Codex) that tells the LLM how the wiki is structured, what the conventions are, and what workflows to follow when ingesting sources, answering questions, or maintaining the wiki. This is the key configuration file — it's what makes the LLM a disciplined wiki maintainer rather than a generic chatbot. You and the LLM co-evolve this over time as you figure out what works for your domain.

Operations

Ingest. You drop a new source into the raw collection and tell the LLM to process it. An example flow: the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log. A single source might touch 10-15 wiki pages. Personally I prefer to ingest sources one at a time and stay involved — I read the summaries, check the updates, and guide the LLM on what to emphasize. But you could also batch-ingest many sources at once with less supervision. It's up to you to develop the workflow that fits your style and document it in the schema for future sessions.

Query. You ask questions against the wiki. The LLM searches for relevant pages, reads them, and synthesizes an answer with citations. Answers can take different forms depending on the question — a markdown page, a comparison table, a slide deck (Marp), a chart (matplotlib), a canvas. The important insight: **good answers can be filed back into the wiki as new pages.** A comparison you asked for, an analysis, a connection you discovered — these are valuable and shouldn't disappear into chat history. This way your explorations compound in the

knowledge base just like ingested sources do.

Lint. Periodically, ask the LLM to health-check the wiki. Look for: contradictions between pages, stale claims that newer sources have superseded, orphan pages with no inbound links, important concepts mentioned but lacking their own page, missing cross-references, data gaps that could be filled with a web search. The LLM is good at suggesting new questions to investigate and new sources to look for. This keeps the wiki healthy as it grows.

Indexing and logging

Two special files help the LLM (and you) navigate the wiki as it grows. They serve different purposes:

index.md is content-oriented. It's a catalog of everything in the wiki — each page listed with a link, a one-line summary, and optionally metadata like date or source count. Organized by category (entities, concepts, sources, etc.). The LLM updates it on every ingest. When answering a query, the LLM reads the index first to find relevant pages, then drills into them. This works surprisingly well at moderate scale (~100 sources, ~hundreds of pages) and avoids the need for embedding-based RAG infrastructure.

log.md is chronological. It's an append-only record of what happened and when — ingests, queries, lint passes. A useful tip: if each entry starts with a consistent prefix (e.g. `## [2026-04-02] ingest | Article Title`), the log becomes parseable with simple unix tools — `grep "^## \[" log.md | tail -5` gives you the last 5 entries. The log gives you a timeline of the wiki's evolution and helps the LLM understand what's been done recently.

Optional: CLI tools

At some point you may want to build small tools that help the LLM operate on the wiki more efficiently. A search engine over the wiki pages is the most obvious one — at small scale the index file is enough, but as the wiki grows you want proper search. [gmd](#) is a good option: it's a local search engine for markdown files with hybrid BM25/vector search and LLM re-ranking, all on-device. It has both a CLI (so the LLM can shell out to it) and an MCP server (so the LLM can use it as a native tool). You could also build something simpler yourself — the LLM can help you vibe-code a naive search script as the need arises.

Tips and tricks

- **Obsidian Web Clipper** is a browser extension that converts web articles to markdown. Very useful for quickly getting sources into your raw collection.
- **Download images locally.** In Obsidian Settings → Files and links, set "Attachment folder path" to a fixed directory (e.g. `raw/assets/`). Then in Settings → Hotkeys, search for "Download" to find "Download attachments for current file" and bind it to a hotkey (e.g. `Ctrl+Shift+D`). After clipping an article, hit the hotkey and all images get downloaded to local disk. This is optional but useful — it lets the LLM view and reference images directly instead of relying on URLs that may break. Note that LLMs can't natively read markdown with inline images in one pass — the workaround is to have the LLM read the text first, then view some or all of the referenced images separately to gain additional context. It's a

bit clunky but works well enough.

- **Obsidian's graph view** is the best way to see the shape of your wiki — what's connected to what, which pages are hubs, which are orphans.
- **Marp** is a markdown-based slide deck format. Obsidian has a plugin for it. Useful for generating presentations directly from wiki content.
- **Dataview** is an Obsidian plugin that runs queries over page frontmatter. If your LLM adds YAML frontmatter to wiki pages (tags, dates, source counts), Dataview can generate dynamic tables and lists.
- The wiki is just a git repo of markdown files. You get version history, branching, and collaboration for free.

Why this works

The tedious part of maintaining a knowledge base is not the reading or the thinking — it's the bookkeeping. Updating cross-references, keeping summaries current, noting when new data contradicts old claims, maintaining consistency across dozens of pages. Humans abandon wikis because the maintenance burden grows faster than the value. LLMs don't get bored, don't forget to update a cross-reference, and can touch 15 files in one pass. The wiki stays maintained because the cost of maintenance is near zero.

The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else.

The idea is related in spirit to Vannevar Bush's Memex (1945) — a personal, curated knowledge store with associative trails between documents. Bush's vision was closer to this than to what the web became: private, actively curated, with the connections between documents as valuable as the documents themselves. The part he couldn't solve was who does the maintenance. The LLM handles that.

Note

This document is intentionally abstract. It describes the idea, not a specific implementation. The exact directory structure, the schema conventions, the page formats, the tooling — all of that will depend on your domain, your preferences, and your LLM of choice. Everything mentioned above is optional and modular — pick what's useful, ignore what isn't. For example: your sources might be text-only, so you don't need image handling at all. Your wiki might be small enough that the index file is all you need, no search engine required. You might not care about slide decks and just want markdown pages. You might want a completely different set of output formats. The right way to use this is to share it with your LLM agent and work together to instantiate a version that fits your needs. The document's only job is to communicate the pattern. Your LLM can figure out the rest.

A Gist That Built The Product

Andrej Karpathy, the man who taught the world how neural networks actually work with his Stanford lectures, who co-founded OpenAI, who built Tesla's Autopilot vision stack, who left OpenAI (again) and has been quietly shipping open-source gold ever since; he dropped a GitHub Gist. Not a repo. Not a framework. Not a library with 47 dependencies and a YAML config that makes you question your career choices.

A Gist.

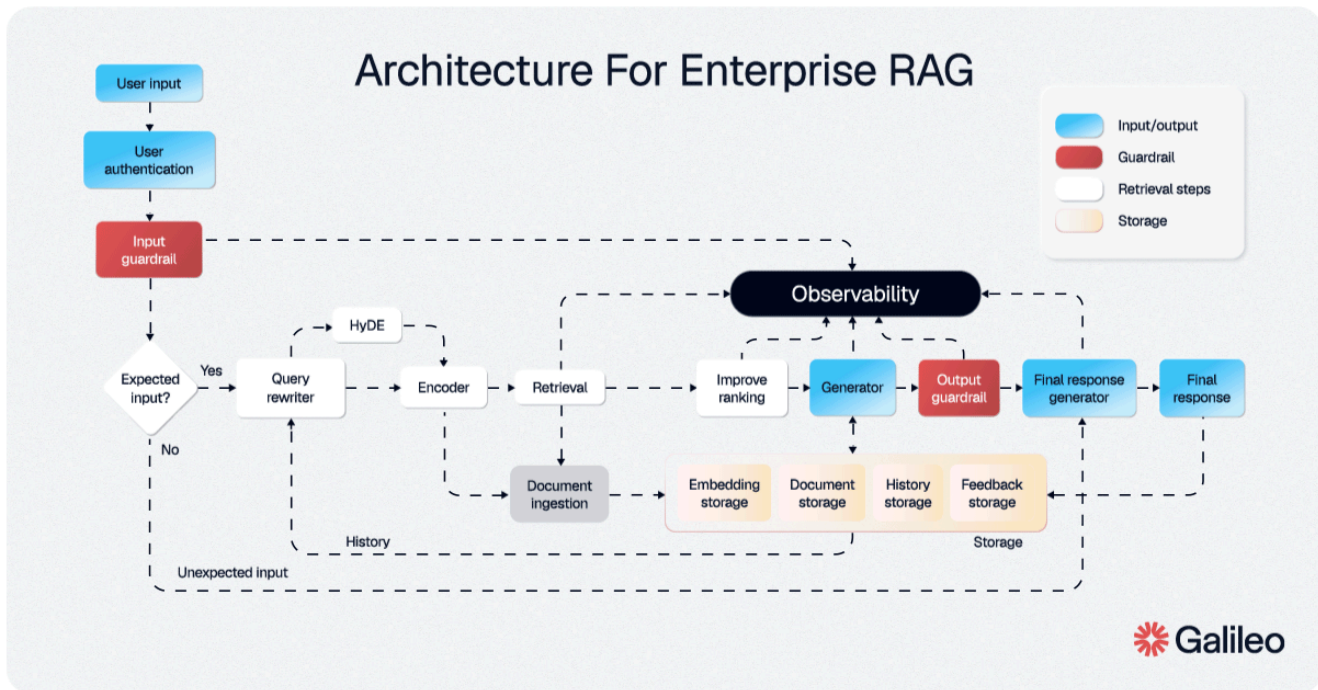
A markdown file.

And the AI community collectively lost it. 5,000+ stars. 1,294 forks. In 48 hours. For a document that contains zero code.

Here's the thing: the document describes something called "LLM Wiki," a pattern for building personal knowledge bases where the LLM doesn't just retrieve information; it compiles, maintains, cross-references, and continuously enriches a structured markdown wiki. The knowledge compounds with every source you add. Nothing disappears into chat history.

And people are calling it the RAG killer.

Let's dig in.



<https://galileo.ai/blog/mastering-rag-how-to-architect-an-enterprise-rag-system>

What RAG Actually Does (and Why People Are Fed Up)

Truth be told, most people's experience with RAG is... underwhelming. Here's the flow everyone knows: you upload documents. The system chunks them into fragments, sometimes intelligently, usually not. Those chunks get embedded into vectors and stored in a database. When you ask a question, the system retrieves the "most similar" chunks, stuffs them into context, and the LLM generates an answer.

It works. Sort of.

The problem is subtle but devastating. Every query is a fresh start. The LLM rediscovers knowledge from scratch every single time. There's no accumulation. No synthesis. No memory of what it figured out last time. Ask the same question tomorrow and the system goes through the identical

retrieval dance, finding the same chunks (if you're lucky) or different ones (if you're not).

But wait.

The chunking problem is even worse than the statelessness.

When you split a 40-page research paper into 512-token fragments, you're destroying context. A paragraph about transformer attention mechanisms gets ripped away from the paragraph that defines the notation. A conclusion references findings from section 3, but section 3 is in a completely different chunk. The embedding might say "this is relevant," but the LLM is reading a sentence with no beginning and no end.

The community has been screaming about this for over a year. Sophisticated chunking strategies, overlapping windows, hierarchical retrieval, re-ranking pipelines; the RAG ecosystem has become a Rube Goldberg machine of workarounds for a fundamental architectural problem.

The information has structure. Chunking destroys that structure. Then we spend enormous engineering effort trying to reconstruct what we already had.

Enter LLM Wiki

The Compiler, before the Search Engine

Karpathy's insight is elegant in its simplicity: what if the LLM didn't retrieve raw documents at query time? What if, instead, it had already read everything, extracted the key information, organized it into a structured wiki

with cross-references and entity pages, and kept the whole thing continuously updated?

The metaphor he uses is perfect: Obsidian is the IDE. The LLM is the programmer. The wiki is the codebase.

You never write the wiki yourself.

You source. You explore. You ask questions. The LLM does all the grunt work.

The architecture has three layers, and their simplicity is what makes the pattern powerful:

Layer 1: Raw Sources

This is your immutable collection of original materials. Articles, papers, transcripts, notes, images. They go into a `raw/` directory and stay there, untouched. Think of them as your source of truth.

Layer 2: The Wiki

This is where the magic happens. The LLM reads raw sources and produces structured markdown files: summaries of each source, encyclopedia-style articles for key concepts and entities, cross-references between related ideas, and a master index that catalogs everything. One source can touch 10 to 15 wiki pages simultaneously. Contradictions between sources get flagged. The synthesis reflects everything the system has ever consumed.

Layer 3: The Schema

A configuration document (Karpathy uses CLAUDE.md) that tells the LLM agent how to behave: what the wiki's structure should look like, how to format pages, what to do during ingestion, how to handle conflicts. It's the constitution the agent operates under.

This isn't the usual RAG pipeline. Not even close.

The Three Operations That Make It Work

The system runs on three core operations, and they form a self-reinforcing loop that gets smarter over time:

Ingest is where sources enter the system.

You drop a new article, paper, or transcript into the raw collection. The LLM reads it, discusses the key takeaways, writes a summary page, updates the master index, and then; here's the critical part; revises every relevant entity and concept page across the wiki. A single paper about transformer efficiency might update pages on attention mechanisms, model compression, inference optimization, and three different researcher entity pages. All automatically. All cross-linked.

Query is how you interrogate the wiki.

You ask a question. The LLM searches the wiki index, pulls up relevant pages, and synthesizes an answer from structured, pre-compiled knowledge. Not fragments. Not chunks. Full, coherent articles that it wrote itself. And here's where it gets really interesting: if the answer is valuable, it becomes a new wiki page. Your exploration compounds in the knowledge base.

Read that again.

Your questions make the wiki smarter.

Lint is the maintenance cycle.

Periodically, the LLM scans the entire wiki for contradictions, stale claims, orphan pages, missing concepts, and data gaps. It's a health check for your knowledge base. The wiki heals itself.

This is his first research pattern since leaving OpenAI that feels genuinely paradigm-shifting. Not because the technology is new; markdown files and LLM agents have existed for a while. But because the framing redefines how we think about knowledge management with AI.

The Vannevar Bush Connection Nobody Is Talking About

Oh!



Karpathy explicitly references Vannevar Bush's Memex from 1945. For those who don't know, Bush was the director of the U.S. Office of Scientific Research and Development during World War II, and he wrote a prophetic essay called "As We May Think" that essentially described the modern internet, personal knowledge management, and hyperlinked information systems; 50 years before the web existed.

How Memex might have worked

by Bob Cotton



Vannevar Bush: Memex virtual prototype

- microfilm capture, storage, display
- use desktop platen to 'scan' new microfiche
- or use head-mounted stereo camera
- microfiche stored in desk drawers
- two microfiche can be projected side by side on screen
- author create notes and 'associative links' between frames
- authors create associative 'trails' for others to follow
- readers browse using joystick, microfilm index, or follow trails



'desktop hypermedia' visualised in 1945

Bush had invented a microfilm reader for Kodak in 1938

the transistor was invented in 1948

first hypertext system (Engelbart) in 1967

the personal computer in 1977

commercial hypermedia software in 1986

Bush's Memex was a hypothetical device where a researcher could store all their books, records, and communications, with "associative trails" linking related ideas across documents. The vision was a personal, curated knowledge store that grew more valuable as you used it.

The problem Bush couldn't solve was maintenance.

Who keeps all those cross-references updated? Who reads every new paper and links it to every relevant existing document? Who flags when a new finding contradicts an old one?

Humans. And humans abandon wikis. Every single time.

But LLMs don't get bored. They don't forget to update the index. They don't skip the cross-referencing because it's Friday afternoon. The tedious bookkeeping that kills every personal knowledge base in practice; that's precisely what LLMs are uniquely good at.

Karpathy didn't just build a better RAG.

He solved Bush's 80-year-old maintenance problem.

If life was this easy, then we would have had it decades ago. But it wasn't. We needed the LLM.

Community Reacted

The discourse around this gist is... fascinating. The thread has been dissected, debated, forked, and meme'd across the internet within hours. And the reactions fall into distinct camps.

The Enthusiasts see this as the future of knowledge work.

People are pointing out that the persistent, compounding nature of the wiki solves the exact wall they've been hitting with traditional RAG: asking the same questions, getting inconsistent answers, never building on previous insights. Some are already building their own implementations.

One developer open-sourced llmwiki.app, connecting directly to Claude via MCP. Another shared a "knowledge synthesis engine" they'd been building independently. Someone literally fed the gist to Hermes and had it start building a wiki for them on the spot.

GitHub - lucasastorian/llmwiki: Open Source Implementation of Karpathy's LLM Wiki. Upload...

Open Source Implementation of Karpathy's LLM Wiki. Upload documents, connect your Claude account via MCP, and have it...

The Skeptics called it reinvention of Cache layer.

They have legitimate points. The sharpest critique? That Karpathy didn't kill RAG; he just renamed the cache layer. And anyone shipping LLM Wiki as the new orthodoxy is going to rediscover deduplication and stale invalidation the hard way in six months.

They also pointed out that Claude Code already uses with its agentic search: models are good at file search now, and letting them search for files with more context beats chunked text search embeddings in many use cases.

The missing piece was compounding memory.

The Pragmatists have perhaps the most useful take: RAG is fine for what it does. The missing piece was compounding memory. Curated notes plus citations plus periodic refresh beats re-retrieving every turn. RAG and LLM Wiki aren't necessarily enemies; they're different tools for different scales and use cases.

And then there's the skeptic who called it "superficial hype," arguing that every time you start Claude Code, the model has only its training knowledge, and reading through skills and configs first doesn't make it an inch smarter.

Umm :(No.

That misses the entire point. The wiki isn't making the model smarter. It's making the knowledge accessible and structured so the model can reason about it effectively. There's a profound difference between stuffing 50 random document chunks into context and giving the model a well-organized encyclopedia it compiled itself.

The Scale Question: 100 Articles Is Not Enterprise

Here's where we need to be honest.

Karpathy reports using this pattern at a scale of approximately 100 articles and roughly 400,000 words. At this size, the model's ability to navigate via summaries and index pages is more than sufficient. The LLM can read the index, identify relevant pages, and pull them into context because modern context windows are large enough to hold an index plus several full articles simultaneously.

What happens at 10^4 , 10^5 , 10^6 files

... A petabyte-scale enterprise knowledge base with compliance requirements, role-based access controls, and 50 agents writing simultaneously?

The enterprise scalability gaps are real and well-documented.

- File-based markdown systems have no RBAC mechanism; you can't restrict agents from sensitive data categories.
- There are no ACID transaction guarantees; multiple simultaneous agents writing to the same wiki pages will produce race conditions.

- There's no tamper-proof audit trail for regulated industries.
- And flat-file systems simply cannot handle the performance demands of large-scale data.

This is not a criticism of Karpathy's work. He explicitly states that the document is intentionally abstract, describing a pattern rather than prescribing an implementation.

The directory structure, schema conventions, and tooling depend on your domain. He knows this is a personal knowledge weapon, not an enterprise platform.

But the tech world buzzes with excitement, and excitement has a way of turning personal tools into overpromised enterprise products. We've seen this movie before.

Why “Just Better RAG” Is the Wrong Frame

Some in the community are dismissing this as “still RAG, just better sorted.”

Not really.

The distinction matters, and it's more than semantic. Traditional RAG is a retrieval operation at query time. You search, you find chunks, you generate. The system is stateless. Every query is day one.

LLM Wiki is a compilation operation at ingest time. When a new source enters the system, the LLM doesn't just index it; it reads it, understands it,

integrates it into existing knowledge, updates cross-references, flags contradictions, and strengthens the synthesis. The knowledge exists in structured, pre-compiled form before you ever ask a question.

This is the difference between a search engine and an encyclopedia.

Google (search) helps you find pages that might contain your answer. Wikipedia (compiled knowledge) gives you a structured article that synthesizes information from hundreds of sources, with cross-references, citations, and editorial oversight.

RAG is the search engine.

LLM Wiki is the encyclopedia.

Both useful. Fundamentally different architectures solving fundamentally different problems.

RAG VS LLM WIKI ARCHITECTURE COMPARISON

DIMENSION	TRADITIONAL RAG	LLM WIKI
When processing happens	Query time (on every question)	Ingest time (once per source, then maintained)
Knowledge state	Stateless; rediscovers from scratch each query	Persistent; compounds with every source added
Document handling	Chunks into fragments, embeds as vectors	Reads in full, compiles structured wiki pages
Context quality	Fragments with lost surrounding context	Full articles the LLM wrote with complete context
Cross-references	None; chunks are independent	Automatic; entity pages link to concept pages
Contradiction handling	None; conflicting chunks may both be retrieved	Flagged during ingest and lint cycles
Infrastructure	Vector database + embedding model + retriever + re-ranker	Markdown files in a folder + LLM agent
Best scale	Thousands to millions of documents	Tens to low thousands of documents
Query feedback loop	None; answers vanish into chat history	Valuable answers become new wiki pages
Maintenance	Manual pipeline tuning, chunk strategy updates	Automated lint cycles by LLM agent
Enterprise readiness	Mature: RBAC, ACID, audit trails available	Early: no RBAC, no concurrency controls, no audit

RAG excels at scale and unpredictable queries. LLM Wiki excels at depth and compounding knowledge in bounded domains.

The Use Cases That Actually Make Sense

Karpathy outlines several use cases, and some are more compelling than others:

Personal knowledge management is the killer app.

Track goals, health metrics, psychology notes. File journal entries alongside research articles. Build a structured picture of yourself over time. This works because the scale is inherently personal (hundreds, not millions of documents) and the value of compounding knowledge is highest when there's one user whose context the system learns over months.

Research synthesis is where this pattern genuinely outperforms RAG.

Reading papers for months, building a comprehensive wiki with an evolving thesis, watching how new findings modify or contradict earlier ones; this is the dream workflow for any researcher. The wiki becomes your externalized understanding, maintained by an agent that never forgets to update the cross-references.

Reading a book is surprisingly powerful.

Build a fan wiki as you read. Characters, themes, plot threads, all cross-referenced, all updated as new chapters reveal new information. For dense fiction or complex non-fiction, this is genuinely useful.

Business operations Manager

This is where it gets interesting and where the scalability concerns bite but still could be perfectly useful. Feeding Slack threads, meeting transcripts, and customer calls into a wiki sounds incredible. The wiki stays current because the AI does the maintenance nobody wants to do. But this is also where you need RBAC, audit trails, and concurrency controls. The gap between “cool pattern” and “production system” is an enterprise engineering moat.

The Obsidian Bet and the Tooling Ecosystem

It's not the size, it's the ecosystem.

Karpathy's choice of Obsidian as the human interface is deliberate and smart. Why reinvent something when it already exists. (Although I have an issue with Obsidian's closed source model... that rant some other day).

Obsidian's graph view reveals structural patterns in the wiki: which concepts are hubs (highly connected), which are orphans (disconnected), where the wiki has dense knowledge clusters and where it has gaps. The Dataview plugin enables dynamic queries against page frontmatter. Marp generates slide decks from wiki content. And because the wiki is just markdown files in a folder, it's automatically a git repository with full version history.

The community is already extending this in interesting directions. Multiple implementations have popped up within days: llmwiki.app, obsidian-wiki integrations, and enterprise teams adapting the pattern for semiconductor knowledge management and service delivery documentation. Someone connected it directly to Claude via MCP servers, meaning you can talk to your wiki from inside your AI agent.

The tooling convergence is real. Local hybrid search tools like qmd provide BM25/vector search with LLM re-ranking. Obsidian Web Clipper converts any web article to markdown with one click. The entire pipeline from "I found an interesting article" to "my wiki is updated with new knowledge" is approaching zero friction.

But if you're about to build your entire company's knowledge infrastructure on markdown files in an Obsidian vault... maybe take a breath first.

LLM WIKI INGEST-COMPILE-QUERY-LINT CYCLE



The Fine-Tuning Endgame Nobody Is Discussing

Here's where it gets really wild.

One of the most underappreciated aspects of Karpathy's pattern is the future direction he hints at: generating synthetic training data from the wiki to fine-tune models. Think about what that means. You spend months building a comprehensive, cross-referenced, contradiction-flagged wiki about your domain. Then you use that wiki to generate training examples. Then you fine-tune a model on those examples.

The knowledge moves from context window to model weights.

Your personal wiki becomes a personal model.

The logical next step is to have the model rebuild its weights with new knowledge. The wiki is the intermediate representation. The compiled knowledge base is a dataset waiting to become a model.

What This Means for the RAG Industry

Let's be clear about something. RAG isn't dead. Not even close. RAG solves real problems at scales where LLM Wiki can't operate: millions of documents, unpredictable queries, real-time data, multi-tenant enterprise deployments with strict security requirements.

But the critique is valid

But the critique is valid. For personal knowledge bases, research projects, small team wikis, and domain-specific knowledge management at the scale of hundreds to low thousands of documents? The LLM Wiki pattern is demonstrably superior.

The pre-compiled structured knowledge eliminates

[Open in app ↗](#)

the chunking problem entirely.

The compounding loop means the system gets better with use. The maintenance automation solves the abandonment problem that kills every personal wiki.

The RAG vendors building billion-dollar businesses on vector databases and retrieval pipelines should be paying attention. Not because LLM Wiki replaces their enterprise products today. But because the pattern exposes a truth the industry has been dodging:

most RAG implementations are over-engineered for what users actually need, and under-engineered for what users actually want.

Users don't want retrieval. They want knowledge. There's a difference.

Innovation never fails to deliver. But sometimes it comes from a single markdown file on GitHub, not from a startup with \$50 million in Series B funding.

The Pattern Is the Product

Here's what I keep coming back to. Karpathy didn't release software. He released a pattern. An idea file. A document you're supposed to hand to your LLM agent and say "build this with me." The implementation details are left deliberately vague because the specifics depend on your domain, your tools, your scale, your preferences.

Some people find this frustrating. They want a repo they can clone, a docker-compose they can run, a SaaS they can sign up for. And those are coming;

the community is already building them.

But the power of publishing a pattern instead of a product is that it invites adaptation rather than adoption. Every implementation will be different because every knowledge domain is different. The semiconductor engineer's wiki won't look like the fiction reader's wiki won't look like the startup founder's wiki. And that's the point.

This is my perspective. You should do what you are comfortable with.

But if you've been frustrated with RAG, if you've felt that your AI tools forget everything the moment the conversation ends, if you've abandoned Notion databases and Obsidian vaults because the maintenance was crushing; Karpathy just gave you a blueprint for making the LLM do the maintenance.

The human curates sources, directs analysis, asks good questions, and thinks about meaning.

The LLM handles everything else.

Vannevar Bush would be proud.

If you have read it until this point, Thank you! You are a hero (and a Nerd ❤️)! I try to keep my readers up to date with "interesting happenings in the AI world," so please 🛎 clap | follow | Subscribe 🛎

Agentic Ai

AI Agent

Generative Ai Tools

Artificial Intelligence

LLM